

# Практическая работа №1

## Теоретическое введение

Prometheus — это система мониторинга и оповещения с открытым исходным кодом, созданная компанией SoundCloud в 2012 году и с тех пор активно развивающаяся сообществом разработчиков. Она предназначена для сбора и анализа временных рядов данных, таких как метрики системы и приложений. Главной особенностью Prometheus является его гибкость и простота использования, что делает его популярным выбором для мониторинга современных распределенных систем.

Основные компоненты Prometheus включают в себя:

Сборщик (Prometheus Server): это ядро системы, которое выполняет сбор, хранение и обработку метрических данных. Он регулярно запрашивает цели мониторинга (например, приложения, серверы, сервисы) для получения метрик и сохраняет их во временной базе данных.

Метрики (Metrics): Prometheus собирает данные в формате временных рядов, называемых метриками. Метрики представляют собой числовые значения, связанные с определенными именованными временными рядами и метками.

Язык запросов (PromQL): для запроса и анализа метрических данных Prometheus предоставляет мощный язык запросов - PromQL. Он позволяет выполнять различные операции, агрегировать данные, применять функции и фильтровать результаты.

Графический интерфейс (Web UI): Prometheus также поставляется с веб-интерфейсом, который позволяет просматривать метрики, создавать графики и дашборды, а также настраивать правила оповещений.

Prometheus широко используется в индустрии благодаря своей простоте установки и настройки, а также возможностям масштабирования

и гибкости. Он позволяет оперативно отслеживать производительность системы, выявлять проблемы и реагировать на них, что делает его важным инструментом для поддержания стабильной и надежной работы приложений и инфраструктуры.

## Полезные ссылки

1. Первые шаги в Prometheus:  
[https://prometheus.io/docs/introduction/first\\_steps/](https://prometheus.io/docs/introduction/first_steps/)
2. Мониторинг показателей хоста Linux с помощью Node exporter:  
<https://prometheus.io/docs/guides/node-exporter/>
3. Обзор Prometheus:  
<https://prometheus.io/docs/introduction/overview/>

## Задание

**Цель работы:** познакомиться с инструментом мониторинга Prometheus.

В данной практической работе вам необходимо скачать и установить Prometheus, а также ознакомиться с его интерфейсом и функциями.

Для установки Prometheus вам понадобится скачать архив с официального репозитория и распаковать его. Сделать это можно, выполнив команды:

```
wget
https://github.com/prometheus/prometheus/releases/download/v2.52.0-rc.0/prometheus-2.52.0-rc.0.linux-amd64.tar.gz

tar xvfz prometheus-*.tar.gz
```

Далее необходимо переместится в директорию, в которую вы распаковали Prometheus. В ней вы можете ознакомиться с файлом prometheus.yml.

Файл prometheus.yml - это основной файл конфигурации сервера Prometheus. Вот общая структура этого файла:

Глобальные настройки (global): Этот блок содержит глобальные настройки, применяемые ко всем целям мониторинга (targets), такие как интервал сбора данных и таймауты. Пример:

```
global:

  scrape_interval: 15s

  evaluation_interval: 15s
```

Настройки мониторинга (scrape\_configs): Этот блок определяет цели мониторинга (targets) и как их собирать. Каждая цель мониторинга

представляет отдельный сервис или приложение, который Prometheus должен опрашивать для получения метрик. Пример:

```
scrape_configs:
  - job_name: 'example-service'
    static_configs:
      - targets: ['example-service:9090']
```

Дополнительные настройки (rule\_files, alerting): В этом блоке можно указать дополнительные файлы правил (rule\_files), которые определяют правила алертинга или другие настройки оповещений (alerting). Пример:

```
rule_files:
  - 'rules/*.rules'
alerting:
  alertmanagers:
    - static_configs:
      - targets:
        - 'alertmanager:9093'
```

Это основная структура файла prometheus.yml. Он может содержать и другие опции, и настройки в зависимости от конкретных требований вашей инфраструктуры и мониторинга. Комментарии в файле prometheus.yml также могут помочь понять, что каждая часть конфигурации делает.

Вот как выглядит файл prometheus.yml по умолчанию:

```
# my global config
global:
```

```
    scrape_interval: 15s # Set the scrape interval
to every 15 seconds. Default is every 1 minute.
```

```
    evaluation_interval: 15s # Evaluate rules every
15 seconds. The default is every 1 minute.
```

```
    # scrape_timeout is set to the global default
(10s).
```

```
# Alertmanager configuration
```

```
alerting:
```

```
  alertmanagers:
```

```
    - static_configs:
```

```
      - targets:
```

```
        # - alertmanager:9093
```

```
    # Load rules once and periodically evaluate them
according to the global 'evaluation_interval'.
```

```
rule_files:
```

```
  # - "first_rules.yml"
```

```
  # - "second_rules.yml"
```

```
    # A scrape configuration containing exactly one
endpoint to scrape:
```

```
    # Here it's Prometheus itself.
```

```
scrape_configs:
```

```
    # The job name is added as a label
`job=<job_name>` to any timeseries scraped from this
config.
```

```
- job_name: prometheus

# metrics_path defaults to '/metrics'

# scheme defaults to 'http'.

static_configs:
  - targets: ["localhost:9090"]
```

По умолчанию Prometheus настроен на получение данных только о собственной работе.

Чтобы запустить Prometheus с нашим только что созданным файлом конфигурации, перейдите в каталог, содержащий двоичный файл Prometheus, и запустите:

```
./prometheus --config.file=prometheus.yml
```

Prometheus должен запуститься. У вас также должна быть возможность перейти на страницу состояния о себе по адресу <http://localhost:9090>. Дайте ему около 30 секунд, чтобы собрать данные о себе из собственной конечной точки HTTP-метрик.

Вы также можете убедиться, что Prometheus предоставляет метрики о себе, перейдя к его собственной конечной точке метрик: <http://localhost:9090/metrics>.

Давайте попробуем взглянуть на некоторые данные, которые Прометей собрал о себе. Чтобы использовать встроенный браузер выражений Prometheus, перейдите по адресу <http://localhost:9090/graph> и выберите представление «Таблица» на вкладке «График».

Как вы можете понять из <http://localhost:9090/metrics>, называется одна метрика, которую Prometheus экспортирует о себе

`promhttp_metric_handler_requests_total`(общее количество `/metrics`запросов, обслуженных сервером Prometheus). Продолжайте и введите это в консоль выражений:

```
promhttp_metric_handler_requests_total
```

Это должно вернуть несколько различных временных рядов (вместе с последним записанным значением для каждого), все с именем метрики `promhttp_metric_handler_requests_total`, но с разными метками. Эти метки обозначают различные статусы запросов.

Если бы нас интересовали только запросы, результатом которых стал HTTP-код 200, мы могли бы использовать этот запрос для получения этой информации:

```
promhttp_metric_handler_requests_total{code="200"}
}
```

Чтобы подсчитать количество возвращенных временных рядов, вы можете написать:

```
count(promhttp_metric_handler_requests_total)
```

Чтобы построить график выражений, перейдите по адресу <http://localhost:9090/graph> и используйте вкладку «График».

Например, введите следующее выражение, чтобы построить график частоты HTTP-запросов в секунду, возвращающих код состояния 200, происходящий в самоочищающемся Prometheus:

```
rate(promhttp_metric_handler_requests_total{code="200"}[1m])
```

Вы можете поэкспериментировать с параметрами диапазона графика и другими настройками.



## **Вопросы к практической работе**

1. Что такое Prometheus и для чего он используется?
2. Какие типы метрик поддерживает Prometheus?
3. Что такое мониторинг цели (target) в Prometheus и какие могут быть типы целей?
4. Какие компоненты входят в архитектуру Prometheus и как они взаимодействуют между собой?
5. Какие типы запросов можно выполнять с помощью языка запросов Prometheus (PromQL)?
6. Какие основные методы сбора метрик поддерживает Prometheus?
7. Какие инструменты используются для визуализации данных в Prometheus?
8. Какие могут быть стратегии масштабирования Prometheus для обработки больших объемов данных?
9. Какие механизмы у Prometheus для обнаружения и регистрации новых целей мониторинга?
10. Какие существуют практические примеры использования Prometheus для мониторинга приложений и инфраструктуры?

## **Критерии оценки**

- Установлен Prometheus
- Освоена работа с интерфейсом и базовым инструментарием Prometheus
- Составлен отчет о проделанной работе со скриншотами её выполнения
- Даны ответы на вопросы к практической работе